Security Foundations for a Multi-Machine Research Fleet

Ronald G. Thomas

2026-05-17

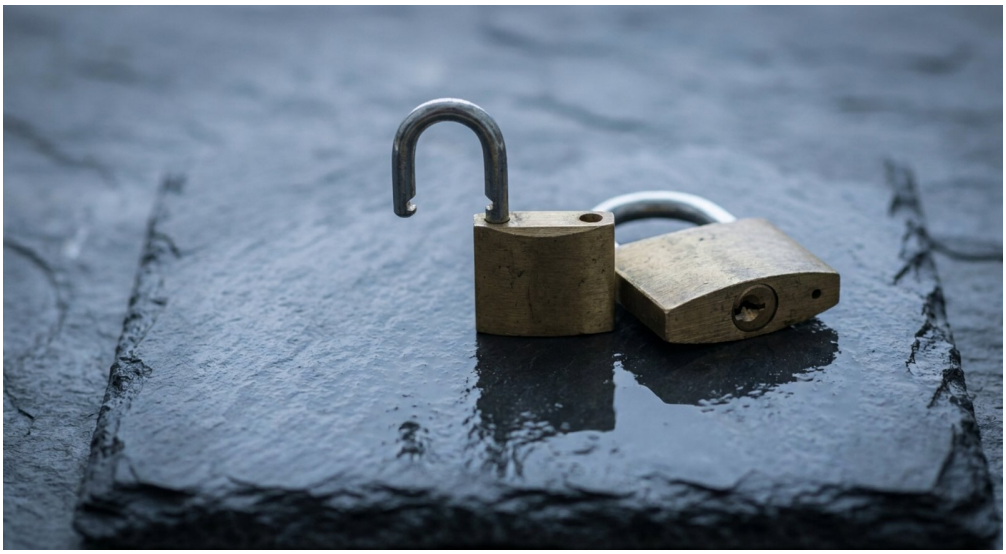


Figure 1: A pair of identical brass padlocks, one open and one locked, resting side by side on a dark stone surface.

A credential store is only as strong as the machine it runs on, the key that unlocks it, and the habits that surround both.

Introduction

A research workflow that spans several machines multiplies both its conveniences and its exposures. Each machine holds a copy of the dotfiles, the project tree, the password store, and the keys that reach the others; each runs scheduled jobs that copy data somewhere else. Security for such a fleet is less a matter of any single tool than of knowing which threats matter, which control answers each, and how to tell, months later, that the controls still hold.

This post is the hub of an eight-part series that documents one such fleet: three Macs, a Linux laptop, and a repurposed laptop that serves as a home server. It states the threat model, lists

the controls and the post that treats each in depth, covers full-disk encryption directly, and gives the quarterly audit checklist that the rest of the series refers back to. The controls described here are the ones in use, and the lessons come from the first full audit of the fleet in the last week of September 2026.

Motivations

- Plaintext secrets accumulate on working machines faster than anyone removes them, and a sync service will copy whatever sits in a synced folder, including private keys.
- Several machines sharing one identity means one compromise reaches all of them unless each relationship has its own credential.
- Backups that report success can still be incomplete, stale, or unreadable, and nothing announces that state.
- Controls that were correct when configured drift as software, folders, and network addresses change.
- A written threat model makes it possible to say what is deliberately out of scope, rather than leaving it undecided.

Objectives

1. State the assets, threats, and out-of-scope risks for a five-machine research fleet.
2. Map each threat to one or more controls, with a pointer to the series post that implements each control.
3. Confirm full-disk encryption on every machine, macOS and Linux.
4. Run a quarterly audit that checks every control on every machine, including whether last night's backups actually succeeded.



Figure 2: A magnifying glass held over a folded paper map showing two marked locations connected by a dotted line.

The Fleet

Name	Role	Operating system
julia	primary workstation	macOS 26
athena	laptop; the only Mac with Time Machine	macOS 26
joe	secondary Mac	macOS 26
mercury	home server (repurposed 2016 MacBook Pro); also a simulation host	Linux Mint
zeus	Linux laptop (ThinkPad); also a second, faster simulation host	Linux Mint 22

All five machines encrypt their disks at rest: FileVault on the Macs, LUKS on the two Linux machines. Machines reach each other by their Tailscale addresses, which stay fixed when the home router hands out new LAN addresses. The LAN is used for the Time Machine share on the home server and, by default on two of the Macs, for the simulation service (with a Tailscale form of each alias for use away from home). Both Linux machines run a default-deny firewall: the home server's has run since late September, and the laptop's was found switched off on 2026-10-11 and turned on then. The home server holds the nightly Borg repositories for the other four machines and the Time Machine share for athena, and it backs up its own files to a repository on julia.

What Is a Threat Model?

A threat model answers three questions: what is worth protecting, from whom or what, and which measures reduce the risk to an acceptable level. Writing it down has a second benefit that is easy to underrate: it records what was deliberately left out, so that an uncovered risk is a decision rather than an oversight.

Assets

- **Credentials:** the GPG key that unlocks the password store, the store itself, SSH private keys, API keys, cloud credentials, and OAuth tokens held by command-line tools.
- **Research data and code:** the project tree (`~/prj`), including unpublished analyses and data covered by use agreements.
- **Backups:** the Borg repositories, cloud copies, and the Time Machine share, which together are the only defense against loss.
- **Configuration:** the dotfiles repository, which encodes how every machine is set up and is therefore a tempting place for a secret to end up by accident.

Threats in Scope

ID	Threat	Typical path
T1	Device loss or theft	A laptop left in a car or a cafe
T2	Account compromise	A reused or phished password; a stolen token
T3	Credential leakage	A settings folder inside a synced folder; a token printed to a terminal or log; a secret committed to git
T4	Ransomware or accidental deletion	A mistaken `rm`, a sync that propagates a deletion, malware encrypting a home folder
T5	Silent backup failure	A job that stops running, skips a folder, or cannot reach its destination, while appearing scheduled
T6	Configuration drift	A fix applied on one machine but not the others; an address or path that changes underneath a script

T3 is the threat that actually materialized in this fleet. A settings folder on two machines had been a symlink into a cloud-sync folder, and for months it carried the GPG private key, backup passphrases, and OAuth tokens to a third party's servers. Whether anyone read them could not be determined, so every exposed credential was replaced. That incident, referred to in this series

as *the exposure*, is told in Migrating Off Dropbox: Beyond Dotfiles, and the rotation discipline it produced is in Secrets Management for the Workflow Construct.

T5 and T6 are the threats most often missed in personal setups. During the first audit they accounted for most of the problems found.

Out of Scope

- Targeted attackers with state-level resources.
- Physical tampering with hardware (evil-maid attacks, hardware implants).
- Supply-chain compromise of the operating system or of Homebrew and distribution packages.
- Compromise of the providers themselves (Google, Apple, Microsoft, GitHub) beyond what encryption of our own data mitigates.

These are real risks; they are excluded because the controls that address them cost far more than the value they would add for a single researcher's fleet.

The Control Set

Each control below answers one or more threats and points to the post that implements it. The audit checklist later in this post checks each one.

Control	Threats	Owning post
C1. Full-disk encryption on every machine	T1	This post
C2. Password store: `pass` with a GPG key	T1, T2, T3	Setting Up pass
C3. Secrets loaded on demand; rotation after any exposure	T2, T3	Secrets Management
C4. No secrets in synced folders; dotfiles in git, not a sync service	T3, T6	Migrating Off Dropbox
C5. One SSH key per relationship; restricted backup keys	T2, T3	This post (summary); Research Backup Architecture
C6. Git hosting over HTTPS through the `gh` credential helper	T2, T3	This post (summary); Secrets Management
C7. Stable Tailscale addressing; default-deny firewall on the server	T2, T6	A Home Server for a Five-Machine Research Fleet
C8. Encrypted 3-2-1 backups, checked daily, restore-tested	T1, T4, T5	Research Backup Architecture
C9. Minimal installed software, recorded declaratively	T6	This post (summary); What Actually Broke

Control	Threats	Owning post
C10. Quarterly audit of every control on every machine	T5, T6	This post

The research-data layer, where an object store with content hashes would add integrity checks that git cannot provide for large files, is treated as a design question rather than a deployed control in Object Storage for Research Data.

C1. Full-Disk Encryption

Full-disk encryption is the control that makes T1 a hardware loss rather than a data breach. On a Mac it is FileVault; on Linux it is LUKS, normally chosen at installation time because adding it later means reinstalling.

```
# macOS: expect "FileVault is On."
fdesetup status

# Linux: expect a crypto_LUKS partition under the root disk
lsblk -o NAME,TYPE,FSTYPE | grep -i luks
```

Encryption protects data only while the machine is off or locked, so it pairs with a short screen-lock delay and a password required immediately on wake (System Settings, Lock Screen, on macOS).

FileVault has one operational consequence that matters for a fleet managed over SSH: after a restart, a FileVault Mac does not unlock its data volume until someone logs in at the keyboard, and until then it refuses SSH logins. A remote restart therefore strands the machine until a person reaches it. Plan restarts accordingly, or accept the trip.

C5. SSH Keys, One per Relationship

The fleet uses a separate SSH key pair for each relationship rather than one key per person or per machine: a key for each git host, a key for interactive logins between machines, one key per machine for its nightly backup connection, and one key per client machine for the simulation service. Losing or exposing one key then affects one relationship.

The backup keys show the pattern at its strongest. On the home server, each backup key is pinned in `authorized_keys` to a forced command that permits only an append-only Borg session against that machine's own repository. A leaked backup key can add data to one repository; it cannot open a shell, read other machines' backups, or delete anything. Because SSH offers every key named in the user's config file, the backup script uses a separate config file of its own so that only the restricted key is presented. The details are in Research Backup Architecture.

The same pattern covers a second service. A client machine that submits simulation jobs to a server has its own key, pinned to a forced command that runs a short gate script. The gate splits the original command into words without any shell and permits exactly two things: the one service program, and a git push into one folder. A leaked key of this kind cannot open a shell, copy files or

forward a port. It can still submit jobs, which run code in a rootless container, so it is narrower than a login key, not harmless. Restricting the key also moved the security boundary into the service program, which had been written for a trusted caller and had to be hardened against hostile input; that story is in Offloading R Simulations to a Home Server. A restricted key is only as safe as the service behind it.

Two settings apply to every key entry: `IdentitiesOnly yes`, so that SSH offers only the key named for that host, and a comment on each public key naming the machine and the relationship, so that a list of authorized keys can be audited by eye.

C6. Git Hosting over HTTPS

Day-to-day git traffic to the hosting service authenticates over HTTPS through the GitHub CLI's credential helper, with the token held in the macOS Keychain, rather than through SSH keys. This keeps one revocable credential per machine, visible on the provider's authorized-applications page, and removes the need to distribute git SSH keys at all. Two practical details: the token needs the `workflow` scope before it can push a change to a GitHub Actions workflow file, and the Keychain is not available inside a plain SSH session, so a `git pull` run on a Mac over SSH fails to authenticate even though the same command works at the keyboard.

C9. Minimal Software

Every installed application is code that runs with the user's privileges and a vendor relationship that can change. The fleet keeps the installed set small, records it in a Brewfile in the dotfiles repository so that a rebuilt machine can be restored with one command, and installs TeX as a per-user TinyTeX with a package list rather than a full distribution. Removing an application completely, including its support files, launch agents, and system extensions, turned out to need more care than dragging it to the Trash; that work is described in What Actually Broke.

Smaller Habits

- **Clipboard.** `pass -c` clears the clipboard after 45 seconds. On Macs signed into the same Apple account, Universal Clipboard can carry a copied password to another device; disable Handoff where that matters.
- **GPG agent cache.** A cache lifetime of minutes rather than hours limits how long an unattended, unlocked session can decrypt secrets. A reference `gpg-agent.conf` is in the post's `analysis/configs/` folder.
- **Keep a copy of the GPG master key offline.** Recommended practice is an offline backup of the master key, kept apart from every daily-use machine, so that losing all machines at once does not lose the password store; see Setting Up pass.
- **Inspect credential files by structure, not content.** Count lines, list key names, or check file types; never print a token to check it. A printed token is treated as exposed and rotated.



Figure 3: A small fireproof document box, lid ajar, with a few folded papers just visible inside, on a stone hearth.

Prerequisites

- Administrative access to every machine in the fleet.
- `pass` and GnuPG installed and initialized; see Setting Up pass.
- SSH access from the workstation to every other machine, so that the audit can be run from one place.
- The backup jobs described in Research Backup Architecture, scheduled with launchd on the Macs and systemd user timers on Linux.

The Quarterly Audit

The audit is where controls are confirmed rather than assumed. The first full run took several days rather than the half hour originally budgeted, mostly because each check turned up something to fix. Later runs should be shorter, but the checklist is written to be run in full, on every machine, including the Linux ones. During the first audit the backup status check initially covered only the Macs, and a Linux laptop's backups had been failing for four nights before anyone looked at it.

Checklist

Encryption and lock (C1)

```
# 1. Disk encryption on every machine
fdesetup status                              # macOS
lsblk -o NAME,TYPE,FSTYPE | grep -i luks     # Linux

# 2. Screen lock requires a password immediately (macOS)
sysadminctl -screenLock status
```

Credentials (C2, C3, C4)

```
# 3. GPG key expiry dates (field 7, epoch seconds; empty means
#    the key never expires); renew anything due within 90 days
gpg --list-keys --with-colons |
  awk -F: '$1 == "pub" || $1 == "sub" { print $1, $5, $7 }'

# 4. Candidate plaintext credential files in the home folder
find ~ -maxdepth 3 \( -name '*.pem' -o -name '.env' \
  -o -name 'credentials' -o -name '*token*' \) \
  -not -path '*/Library/*' 2>/dev/null

# 5. The password store is not inside a synced folder
cd ~/.password-store && pwd -P

# 6. The store's git remote is self-hosted
git -C ~/.password-store remote -v

# 7. Stale entries: review the tree, remove what is no longer used
pass ls
```

Keys and authorizations (C5, C6)

```
# 8. Authorized SSH keys on every machine: each comment names a
#    machine and a relationship; no unknown entries
cat ~/.ssh/authorized_keys | awk '{ print $NF }'
#    Restricted entries (backup, simulation) start with 'restrict' and
#    a command= that names a program that exists; list them, and check
#    that no machine has an unrestricted key it no longer needs
grep -E '^(restrict|command=)' ~/.ssh/authorized_keys | cut -c1-110

# 9. Keys and OAuth apps on each hosting account: review in the
#    provider's settings pages; remove anything not recognized

# 10. The gh login is in the Keychain, not a plain file, and its
#     scopes are what you expect
gh auth status
```

Dotfiles and drift (C4, C6, C9)

```
# 11. Nothing sensitive staged or tracked in the dotfiles repository
git -C ~/dotfiles status --short
git -C ~/dotfiles ls-files | grep -i -E 'token|secret|hosts\.yml'

# 12. Every machine is on the same dotfiles commit
git -C ~/dotfiles log -1 --oneline

# 13. The installed app set matches the Brewfile
brew bundle check --file ~/dotfiles/Brewfile
```

Backups (C8), on every machine

```
# 14. Last exit status of every scheduled backup job (macOS);
#     the middle column must be 0
launchctl list | grep -E 'local\.(borg|rclone)'

# 15. The same on Linux
systemctl --user list-timers --all | grep -i borg
systemctl --user show -p Result -p ExecMainStatus \
  borg-backup.service

# 16. Log freshness: last night's log exists and ends in success
ls -t ~/.borg/log | head -1
tail -3 "$HOME/.borg/log/$(ls -t ~/.borg/log | head -1)"

# 17. Restore test: extract one known file from the latest archive
#     into a scratch folder and compare it with the original
#     (see the backup post for the repository variables)
borg extract --dry-run --list "::$(borg list --last 1 --short)" \
  | head
```

Network and server (C7)

```
# 18. The home server's firewall is active and default-deny
sudo ufw status verbose

# 19. Services listen where they should, and only there
sudo ss -tlnp
```

Items 1 to 16 take a few minutes per machine once the fleet is in good order. Item 17 is the one most often skipped and the one that matters most: a backup that has never been restored is an assumption. A full restore test, onto a spare disk, belongs on the checklist once a year.

Why the Backup Items Exist

Every backup problem found during the first audit was found by reading a log or an exit status by hand, and in every case the information had been sitting there for days. A nightly job silently

skipped three folders after they moved; a cloud job had never run at all; and Borg failed for several nights on three machines after the home server received new LAN addresses at a reboot. The scheduler's own record of each job's last exit status already showed the failure. Items 14 to 16 make reading that record a routine, and a daily automated version of the same check is the natural next step.

Things to Watch Out For

1. **A restarted FileVault Mac is unreachable over SSH.** Symptom: SSH times out or is refused after a remote restart. Fix: log in once at the keyboard; schedule restarts for when someone is present.
2. **A status check that skips a machine.** Symptom: everything looks green, but one machine has not been examined. Fix: run every audit item on every machine, Linux included, from a fixed list of hosts.
3. **A settings folder that is secretly a symlink.** Symptom: a secret appears in a sync service or a git working tree. Fix: before trusting a folder, check it with `ls -la`, and audit `$HOME` for symlinks into synced locations; see Migrating Off Dropbox.
4. **Revoking one OAuth authorization revokes it everywhere.** Symptom: every machine is logged out of a tool at once. Fix: plan a revocation as an all-machines action and re-authenticate each one.
5. **Keychain credentials are absent in SSH sessions.** Symptom: `git pull` over SSH into a Mac asks for a username. Fix: run git operations that need the hosting token at the keyboard, or accept that remote sessions are read-only for that purpose.
6. **Printing a file to check it.** Symptom: a live token on screen or in a scrollback buffer. Fix: rotate it, then inspect credential files by structure only.
7. **A scheduled job firing before the network is up.** Symptom: a 02:00 job on a sleeping Mac fails with a connection timeout while later jobs succeed. Fix: have the script wait or retry for connectivity before declaring failure.

Uninstall / Rollback

The controls in this post are configuration rather than software, and most are reversed in the same place they were set: FileVault in System Settings (decryption runs in the background and can take hours), the screen-lock delay in Lock Screen settings, the GPG agent cache in `~/.gnupg/gpg-agent.conf` followed by `gpgconf --kill gpg-agent`. Reversing LUKS requires reinstalling. The controls owned by other posts each have their own rollback section.



Figure 4: A single chain-link fence post with a padlock and chain wrapped around it.

What Did We Learn?

Lessons Learned

- **The threat that arrives is rarely the one planned for.** The original model centered on device theft. What happened was leakage through a sync service, a quieter failure that a stolen laptop's encryption does nothing to stop.
- **Controls decay without anyone touching them.** Folders move, addresses change, a job drops out of the scheduler after a restart. None of these announce themselves; only a check does.
- **Read logs and exit statuses; do not trust schedules.** A job being scheduled says nothing about whether it ran or succeeded.
- **A fix is not done until it is verified on every machine it touches.** One fix to the backup scripts introduced a dependency that one machine lacked, and broke that machine's jobs entirely.
- **Treat exposure as compromise.** When it cannot be shown that a leaked secret was not read, replace it, then test that the old one no longer works. Some revoked-in-principle credentials were still live when tested.

Limitations

- The fleet has one operator. The model does not address shared administration, role separation, or onboarding another person.

- Hardware security keys are not used; private keys live on disk, protected by passphrases and full-disk encryption.
- Backup monitoring is a manual check on the audit schedule and on demand, not an automated daily alert.
- The audit checklist is a set of commands rather than a single tested script covering both macOS and Linux.

Opportunities for Improvement

1. **A daily backup status report.** Collect each job's last exit status and log age from every machine and flag anything not successful or older than a day.
2. **Hardware-backed keys.** Move SSH and GPG private keys onto a hardware token so that a copied file is no longer a usable key.
3. **Automated secret scanning.** A pre-commit hook on the dotfiles and project repositories would catch the next secret before it is committed.
4. **Short-lived cloud credentials.** Replace long-lived access keys with credentials issued per session where the provider supports it.
5. **One audit script for both platforms.** Turn the checklist into a script that runs on every host over SSH and prints a single report.

Wrapping Up

A threat model is useful in proportion to how honestly it is kept. For this fleet, the planned threat was a lost laptop and the actual one was a sync service quietly carrying keys off the machine, followed by a run of backups that looked fine and were not. The controls in this series answer both, but they hold only as long as someone checks them.

Key takeaways:

- Write down what is out of scope; it turns gaps into decisions.
- Give every relationship its own credential, and restrict the ones that run unattended.
- Encrypt every disk, every backup, and every Time Machine destination.
- Check exit statuses and logs on every machine, Linux included, and test a restore.
- Verify each fix on every machine it touches.

The Series

1. **Security Foundations for a Multi-Machine Research Fleet** (this post): threat model, controls, full-disk encryption, and the quarterly audit.
2. Setting Up pass: a Unix Password Manager: the GPG-encrypted password store and its use across machines.
3. Secrets Management for the Workflow Construct: API keys, cloud credentials, OAuth tokens, and rotation.
4. Migrating Off Dropbox: Beyond Dotfiles: leaving a sync daemon, and the exposure that forced a rotation.

5. Research Backup Architecture: Borg, Cloud Copies, and Time Machine: the backup tiers, retention, monitoring, and restore testing.
6. A Home Server for a Five-Machine Research Fleet: the server's roles, networking, firewall, and storage.
7. Object Storage for Research Data: The Layer Git Cannot Hold: a design argument for content-addressed research data.
8. Running the Quarterly Security Audit for the First Time: What Actually Broke: the first audit, told as it happened.

See Also

Security series:

1. Security Foundations for a Multi-Machine Research Fleet (this post)
2. Setting Up pass: a Unix Password Manager
3. Secrets Management for the Workflow Construct
4. Migrating Off Dropbox: Beyond Dotfiles
5. Research Backup Architecture: Borg, Cloud Copies, and Time Machine
6. A Home Server for a Five-Machine Research Fleet
7. Object Storage for Research Data: The Layer Git Cannot Hold
8. Running the Quarterly Security Audit for the First Time: What Actually Broke

External references:

- OWASP Threat Modeling: a general introduction to the method.
- Apple Platform Security: FileVault: how FileVault protects the data volume.
- cryptsetup and LUKS: the Linux full-disk encryption reference.
- GnuPG documentation: key management, subkeys, and agent configuration.
- Tailscale documentation: the overlay network used for inter-machine addressing.
- Have I Been Pwned: a periodic check of whether an account has appeared in a known breach.

Reproducibility

Tested configuration (verified 2026-10-01):

Component	Version
macOS	26.6
Linux Mint	22.3
GnuPG	2.5
pass	1.7.4
OpenSSH	10.3
gh CLI	2.101
Tailscale	1.102
Borg	1.4.5

Configuration artifacts:

- `analysis/configs/security-audit.sh`: a runnable macOS subset of an earlier version of the checklist; the checklist in this post is the current reference.
- `analysis/configs/gpg-agent.conf`: a reference agent configuration with `pinentry-mac` and short cache lifetimes.

Items 1, 3, 13, and 14 were run as written on julia (macOS), and items 1 and 15 on zeus (Linux), on 2026-10-01. The authorized-keys listing in item 8 was run on the home server and on zeus on 2026-10-12, and showed the restricted backup and simulation entries; the `grep` line added to item 8 was not run in exactly that form. The remaining items are the commands used during the first audit, collected here; they have not yet been run end to end as a single pass in this exact form. Item 9 is a manual review of provider settings pages, and item 17 needs the repository variables described in the backup post.

Let's Connect

Questions, suggestions, or corrections are welcome.

- **GitHub:** rgt47
- **Twitter/X:** @rgt47
- **LinkedIn:** Ronald Glenn Thomas
- **Email:** Contact form

I would like to hear from you if:

- You find an error, or a better approach to anything in this series.
- You run a small research fleet and have found where this design breaks.
- You want to compare notes on backups, secrets, or reproducible research infrastructure.